PYQ Knowledge Layer — AI Agent Integration Specification

Implementation-ready specification for adding Concepts, Entities and Relationships to the existing PYQ tagging pipeline without changing the existing taxonomy.

1. Objective

Goal: improve the platform's question-specific “Similar Questions” retrieval. The current application already tags questions using Subject, Microtheme, Nano Theme and Difficulty. Keep those fields and extraction behavior intact. Add a separate **Knowledge Layer** consisting of Concepts, Entities and Relationships.

The Knowledge Layer is not a replacement taxonomy. It is an additional retrieval layer that allows the system to distinguish questions about a precise concept such as **Article 15** from other questions that merely share the same broad Nano Theme.

The supplied source demonstrates the existing format: for example, Q1 contains Subject, Microtheme, Nano Theme and Difficulty, followed by question, answer and solution content. ■filecite■turn0file0■L2-L20■

2. Non-Negotiable Compatibility Rules

Rule	Requirement
Existing fields	Do not remove, rename, reinterpret or restructure Subject, Microtheme, Nano Theme or Difficulty.
Existing pipeline	Treat the Knowledge Layer as an additional enrichment stage after existing extraction.
Existing filters	Exam Year, Subject, Microtheme/Nano Theme and other existing filters continue to work exactly as before.
No forced AI dependency	The retrieval system must work with deterministic metadata, database queries and weighted scoring. AI may be added later, but is not required.
Optional migration	Existing questions can be enriched progressively. Missing Knowledge Layer data must not break existing questions.

3. Existing Record + New Knowledge Layer

The target record should conceptually look like this:

```
{
  "subject": "...",
  "microtheme": "...",
  "nano_theme": "...",
  "difficulty": "...",

  "concepts": [
    "..."
  ],

  "entities": [
    "..."
  ],

  "relationships": [
    {
      "source": "...",
      "relation": "...",
      "target": "..."
    }
  ]
}
```

4. Knowledge Layer Definitions

Field	Definition	Example
Concepts	Abstract ideas, themes, principles or topics tested by the question.	Right to Equality; Reservation; Discrimination
Entities	Specific identifiable people, acts, articles, institutions, events, texts, places, committees, schemes, etc.	Article 15; Government of India Act 1935; Mahatma Gandhi

Field	Definition	Example
Relationships	A directed or undirected semantic connection between two concepts/entities.	Article 15 → Reservation

5. What Counts as a Concept

Use Concepts for ideas rather than unique named objects. Examples: Right to Equality, Constitutional Development, Women's Suffrage, Mughal Administration, Prison Literature, Quit India Movement, Reservation, Sanskrit Drama.

Do not create a new Concept merely because a named entity appears. For example, “Article 15” should normally be an Entity; “Prohibition of Discrimination” and “Reservation” can be Concepts.

6. What Counts as an Entity

Use Entities for stable, identifiable nodes that can be reused across questions. Suggested entity types include: CONSTITUTION_ARTICLE, ACT, CASE, PERSON, ORGANIZATION, EVENT, MOVEMENT, BOOK, INSTITUTION, PLACE, COMMITTEE, SCHEME, ADMINISTRATIVE_UNIT and OTHER.

An entity should have a canonical name and aliases where applicable. Example: canonical entity “Article 15”; aliases may include “Art. 15”, “Art 15”, “Article fifteen”.

For constitutional provisions, sub-provisions can be separate entities where useful: Article 15, Article 15(1), Article 15(3), Article 15(4), Article 15(5), Article 15(6).

7. Relationship Model

Relationships should be explicit, reusable and directional when direction matters.

```
{
  "source": "Article 15(4)",
  "relation": "provides_for",
  "target": "Special provisions for SEBC/SC/ST"
}
```

Recommended relation vocabulary should remain controlled rather than allowing unlimited free-text relation names. Suggested relations: related_to, part_of, provides_for, prohibits, associated_with, founded_by, written_by, adopted_by, occurred_at, occurred_in, succeeded_by, predecessor_of, resulted_in, compared_with, member_of, officer_of, headed_by, divided_into, translated_by.

8. Enrichment Pipeline

```
EXISTING QUESTION EXTRACTION
        |
        +--> Subject
        +--> Microtheme
        +--> Nano Theme
        +--> Difficulty
        +--> Question / Options / Answer / Solution
        |
        v
KNOWLEDGE-LAYER ENRICHMENT
        |
        +--> Concept extraction
        +--> Entity extraction
        +--> Entity normalization / alias resolution
        +--> Relationship mapping
        |
        v
STORE EXTENDED QUESTION RECORD
        |
        v
SIMILAR-QUESTION RETRIEVAL INDEX / QUERIES
```

The source material shows that solutions often contain strong signals for the Knowledge Layer. For example, Q2 explicitly lists related Acts and distinguishes the 1919 and 1935 Acts; Q3 links the Quit India Resolution, AICC and Cripps Mission; Q6 provides institution/person relationships; and Q7 provides INA/officer/trial relationships. ■filecite■turn0file0■L53-L67■ ■filecite■turn0file0■L68-L97■ ■filecite■turn0file0■L153-L186■ ■filecite■turn0file0■L188-L217■

9. Input Material for Enrichment

The enrichment stage should consider, where available:

```
1. Question text
2. Options
3. Correct answer
4. Solution / explanation
5. Existing Subject
6. Existing Microtheme
7. Existing Nano Theme
8. Existing related articles / acts / cases / references
9. Existing source/reference metadata
```

Do not require every input component to be present. The enrichment system should degrade gracefully.

10. Article 15 Target Example

For a question specifically about Article 15, the extended record can be:

```
[Subject: Polity]
[Microtheme: Fundamental Rights]
[Nano Theme: Right to Equality]
[Difficulty: Moderate]

[Concepts:
  Right to Equality,
  Prohibition of Discrimination,
  Reservation,
  Special Provisions]

[Entities:
  Article 15,
  Article 15(1),
  Article 15(3),
  Article 15(4),
  Article 15(5),
  Article 15(6)]

[Relationships:
  Article 15 -> Right to Equality,
  Article 15 -> Prohibition of Discrimination,
  Article 15(3) -> Special Provisions for Women and Children,
  Article 15(4) -> Special Provisions for SEBC/SC/ST,
  Article 15(5) -> Educational Institutions,
  Article 15(6) -> EWS Reservation]
```

A different question may have the same Nano Theme but only Article 14. It should therefore rank below an Article 15 question when the current question is specifically about Article 15.

11. Recommended Database Model

```
questions
  id
  exam_id
  exam_year
  subject_id
  microtheme_id
  nano_theme_id
  difficulty
  question_text
  options
  answer
  solution
  ...

concepts
  id
  canonical_name
  type
  aliases
  status

entities
  id
  canonical_name
  type
  aliases
  parent_entity_id
  status

question_concepts
  question_id
  concept_id
```

```
  weight

question_entities
  question_id
  entity_id
  relevance

relationships
  id
  source_entity_id
  relation_type
  target_entity_id
  confidence
  source_question_id

question_relationships
  question_id
  relationship_id
```

If the current application uses a document database instead, the same logical model can be embedded as arrays. Do not force a database migration solely for this feature.

12. Canonicalization / Deduplication

The same concept/entity must not appear as multiple independent nodes because of spelling differences. Maintain canonical names and aliases.

```
"Article 15"
aliases:
  - "Art 15"
  - "Art. 15"
  - "Article fifteen"

"Government of India Act 1935"
aliases:
  - "GoI Act 1935"
  - "Government of India Act, 1935"
  - "GOI Act 1935"
```

Canonicalization should happen before similarity retrieval.

13. Similar-Question Retrieval Algorithm

Use a deterministic candidate-generation and ranking pipeline.

```
CURRENT QUESTION
      |
      +--> exact entity IDs
      +--> concept IDs
      +--> relationship targets
      +--> existing Nano Theme
      +--> existing Microtheme
      |
      v
CANDIDATE QUESTIONS
      |
      v
WEIGHTED SCORE
      |
      v
FILTER / DEDUPLICATE
      |
      v
TOP N RELATED QUESTIONS
```

14. Initial Scoring Model

Signal	Initial weight	Meaning
Exact entity match	50	Same canonical entity, e.g. Article 15
Specific provision match	35	Same Article 15(4), Act section, etc.
Concept match	20	Same meaningful concept
Relationship match	15	Connected through configured relationship

Signal	Initial weight	Meaning
Same Nano Theme	10	Same existing fine-grained taxonomy
Same Microtheme	5	Broad contextual similarity

These are starting values. Store weights/configuration outside application code if possible so they can be tuned using user behavior later.

15. Candidate Retrieval Strategy

Do not compare every question with every other question. Retrieve candidates using indexed IDs.

```
Candidate set =
  questions sharing any exact entity
  UNION
  questions sharing any concept
  UNION
  questions connected through configured relationships
  UNION
  questions sharing Nano Theme
  UNION
  questions sharing Microtheme
```

Then calculate the score only for this candidate set. Exclude the current question itself.

16. Example: Article 15

```
CURRENT QUESTION
Entities:
  Article 15
  Article 15(4)

Concepts:
  Right to Equality
  Reservation
  Discrimination

Candidate A:
  Article 15
  Article 15(4)
  Reservation
  => very high score

Candidate B:
  Article 15
  Discrimination
  => high score

Candidate C:
  Article 16(4)
  Reservation
  => medium score

Candidate D:
  Article 14
  Right to Equality
  => lower score

Candidate E:
  same Nano Theme only
  => fallback score
```

This gives the user questions related to the exact concept first, then progressively broader related questions.

17. Retrieval API Contract

```
GET /questions/{questionId}/similar

Query parameters:
  limit=5
  min_score=0
  include_related=true

Response:
{
  "question_id": 123,
  "results": [
```

```
    {
      "question_id": 456,
      "score": 85,
      "match_reasons": [
        "Exact entity: Article 15",
        "Concept: Reservation",
        "Relationship: Article 15 -> Reservation"
      ]
    }
  ]
}
```

The API should return match_reasons for explainability. This is useful for debugging and can optionally be shown in the UI.

18. UI Behavior

Keep the top-level filters unchanged: Exam Year, Subject, Topic/Microtheme, Sub-topic/Nano Theme. When the user opens a question, the Similar Questions component should use the Knowledge Layer.

```
Similar Questions
-----------------
1. Exact concept matches
2. Closely related concepts
3. Same Nano Theme
4. Same Microtheme
```

Optionally display a small reason label such as “Article 15”, “Reservation”, or “Same Nano Theme”. Avoid exposing internal scores to users.

19. Migration Strategy

```
Phase 1
  Add nullable concepts/entities/relationships fields.

Phase 2
  Create Concept and Entity masters.

Phase 3
  Enrich new questions automatically.

Phase 4
  Backfill high-value existing PYQs.

Phase 5
  Enable similarity retrieval.

Phase 6
  Tune weights using clicks, attempts and subsequent-question behavior.
```

Existing questions without Knowledge Layer metadata should fall back to Nano Theme/Microtheme similarity.

20. Quality Controls

Add validation before publishing enriched records:

```
- No duplicate canonical entities
- No empty relationship source/target
- Relation type must be from controlled vocabulary
- Entity aliases must resolve to one canonical entity
- Concept names must be normalized
- Current question must not appear in its own recommendations
- Do not recommend duplicate question IDs
- Preserve existing taxonomy fields
- Missing knowledge metadata must not fail question ingestion
```

21. Non-AI First, AI Optional Later

The initial implementation should be fully functional without an LLM. Deterministic rules, canonical dictionaries, controlled vocabularies, relationship maps, full-text search and weighted database retrieval are sufficient.

If AI is introduced later, it should be an enrichment or ranking enhancement rather than a hard dependency. The Knowledge Layer should remain the stable contract between ingestion and retrieval.

22. Example Extended Records from the Supplied 2021 GS1 File

Q1 already identifies Bhavabhuti, Hastimalla and Kshemeshvara as playwrights and names their literary works in the solution. ■filecite■turn0file0■L3-L20■

```
{
  "subject": "History & Culture",
  "microtheme": "SourcesXLiterature",
  "nano_theme": "Bhavabhuti, Hastimalla, Kshemeshvara — playwrights",
  "difficulty": "Tough",
  "concepts": [
    "Ancient Indian literature",
    "Sanskrit drama",
    "Playwrights"
  ],
  "entities": [
    "Bhavabhuti",
    "Hastimalla",
    "Kshemeshvara",
    "Mahaviracharita",
    "Purvapurana",
    "Dasavatar Charita"
  ],
  "relationships": [
    {"source":"Bhavabhuti","relation":"playwright","target":"Sanskrit drama"},
    {"source":"Hastimalla","relation":"playwright","target":"Purvapurana"}
  ]
}
```

Q5 provides a clear administrative hierarchy: Pargana → Sarkar → Suba, with associated officials and units, making it a good example of entity-to-entity relationships. ■filecite■turn0file0■L126-L152■

```
{
  "subject": "Medieval History",
  "microtheme": "Medieval Administration",
  "nano_theme": "Paragana–Sarkar–Suba — administrative units",
  "difficulty": "Easy",
  "concepts": [
    "Mughal administration",
    "Administrative hierarchy",
    "Revenue administration"
  ],
  "entities": [
    "Pargana",
    "Sarkar",
    "Suba",
    "Subedar",
    "Faujdar",
    "Diwan"
  ],
  "relationships": [
    {"source":"Suba","relation":"divided_into","target":"Sarkar"},
    {"source":"Sarkar","relation":"divided_into","target":"Pargana"},
    {"source":"Suba","relation":"headed_by","target":"Subedar"},
    {"source":"Sarkar","relation":"headed_by","target":"Faujdar"}
  ]
}
```

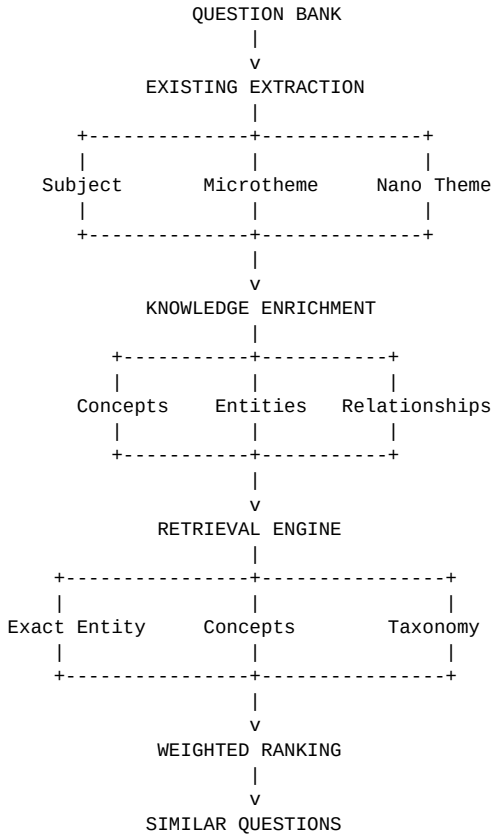
23. Agent Instructions — What the Coding Agent Should Implement

The following is the concise implementation directive for an AI coding agent:

```
TASK:
Integrate a Knowledge Layer into the existing PYQ platform.

DO NOT:
- change existing Subject/Microtheme/Nano Theme/Difficulty fields
- change existing question filtering behavior
- make AI/LLM mandatory
- replace the current extraction pipeline

ADD:
1. concepts[]
2. entities[]
3. relationships[]

IMPLEMENT:
- Concept master
- Entity master with canonical names + aliases
- Controlled relationship types
- Question-to-concept mapping
- Question-to-entity mapping
- Relationship mapping
```

```
- Deterministic enrichment stage
- Similar-question candidate retrieval
- Weighted similarity scoring
- Top-N related-question API
- Match reasons for explainability
- Fallback to existing Nano Theme/Microtheme when metadata is missing
- Validation and deduplication
- Backfill/migration support

PRIORITY:
Exact entity > specific provision > concept > relationship > Nano Theme > Microtheme.

SUCCESS CONDITION:
When a user solves a question about a specific concept/entity such as Article 15,
the system should return questions connected specifically to Article 15 first,
followed by closely related concepts and finally broader taxonomy matches,
without requiring an LLM at runtime.
```

24. Final Architecture



25. Definition of Done

The integration is complete when: (1) existing questions continue to load unchanged; (2) new questions can carry the three Knowledge Layer fields; (3) canonical Concepts and Entities can be reused across questions; (4) relationships are queryable; (5) similar-question retrieval prioritizes exact entities/concepts over broad taxonomy; (6) the API returns ranked results and match reasons; (7) questions without new metadata still receive fallback recommendations; and (8) the feature operates without an AI/LLM dependency.